MACHINE LEARNING LABORATORY

OBSERVATION NOTEBOOK

FIND-S Algorithm

Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples.

Exp.	Title	Dataset
1	Basic implementation - EnjoySport	EnjoySport
2	CSV input - Student placement	Student Placement (CSV)
3	Step-by-step trace with graph	EnjoySport (trace + graph)
4	Class-based implementation - Flu diagnosis	Flu Diagnosis
5	Prediction, accuracy and limitations	Loan Approval

Name	JOEL BENN JOHN B
Register No.	1911260003
Department / Year	CSE
Course Code	ITA0614
Academic Year	2026-2030

MACHINE LEARNING LABORATORY | EXPERIMENT 1 OF 5

FIND-S: Basic Implementation (EnjoySport Dataset)

Evaluation Component	Max. Marks	Marks Awarded
Aim & Concept	5	
Algorithm & Procedure	10	
Program (logic, readability)	15	
Output & Observation	10	
Result & Conclusion	5	
Viva-voce	5	
TOTAL	**50**	
Staff Signature with Date:		

1. Experiment No. & Date

Experiment No.	1 (Part 1 of 5)
Date	____ / ____ / _____
Topic	Basic implementation - EnjoySport

2. Experiment Question

> **Question**
> Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Part 1: Basic implementation - EnjoySport]

3. Aim

To implement the FIND-S algorithm in Python and to find the maximally specific hypothesis for the EnjoySport training data using a hard-coded list of training examples.

4. Concept

Concept learning is the task of inferring a Boolean-valued function (the target concept) from labelled training examples. Hypotheses are written as a conjunction of attribute constraints.

FIND-S starts with the most specific hypothesis and generalises it only when a positive example is not covered. It considers positive examples only, never moves to a more specific hypothesis, and assumes the training data is noise-free and that the target concept is contained in the hypothesis space H.

Hypothesis symbols: '?' accepts any value; a specific value (e.g. Sunny) accepts only that value; '0' / ∅ accepts no value.

5. Dataset Description

Source	Hard-coded Python list
Attributes	Sky, AirTemp, Humidity, Wind, Water, Forecast
Target attribute	EnjoySport (Yes = positive, No = negative)
Summary	4 instances, 6 attributes, 3 positive and 1 negative example (Mitchell, Machine Learning, Table 2.2).

Training data *(shaded rows = positive examples, unshaded rows = negative examples)*

No.	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
2	Sunny	Warm	High	Strong	Warm	Same	**Yes**
3	Rainy	Cold	High	Strong	Warm	Change	**No**
4	Sunny	Warm	High	Strong	Cool	Change	**Yes**

6. Algorithm

Initialise h to the most specific hypothesis in H: h = <∅, ∅, ∅, ..., ∅> (written as '0' in the program).

For each training example x in the training data:

If x is a positive example (target = Yes), then for each attribute a_i in h:

(a) If h[i] = '0', set h[i] = x[i] (copy the first positive example).

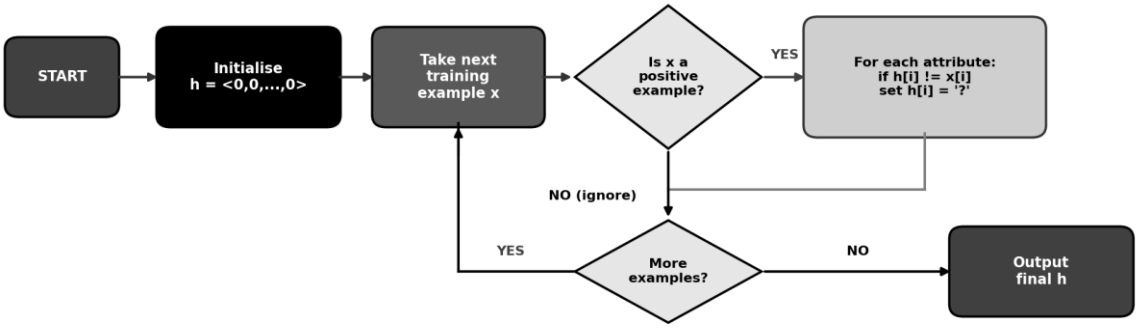
(b) Else if h[i] = x[i], leave h[i] unchanged.

(c) Else replace h[i] with '?' (generalise just enough to cover x).

If x is a negative example, ignore it (h is not changed).

Output the hypothesis h (the maximally specific hypothesis consistent with all positive examples).

FIND-S Algorithm - Flowchart



7. Step-by-step Procedure

1. Import the required modules and define the attribute names and the training data set.
2. Initialise the hypothesis with the most specific value (all '0'), or copy the first positive example.
3. Take the training examples one by one. If an example is negative, skip it.

4. For a positive example, compare each attribute of h with the example: keep the value if they match, otherwise replace it with '?'.
5. Record the hypothesis after every example (see the trace table below).
6. Print the final hypothesis after all examples have been processed.

Hypothesis trace *(shaded rows = positive examples)*

Step	Instance	Label	Hypothesis h after step	Action
Initial	-	-	**<0, 0, 0, 0, 0, 0>**	-
x1	Sunny, Warm, Normal, Strong, Warm, Same	Yes	**<Sunny, Warm, Normal, Strong, Warm, Same>**	First positive - copy
x2	Sunny, Warm, High, Strong, Warm, Same	Yes	**<Sunny, Warm, ?, Strong, Warm, Same>**	Generalised
x3	Rainy, Cold, High, Strong, Warm, Change	No	**<Sunny, Warm, ?, Strong, Warm, Same>**	Negative - ignored
x4	Sunny, Warm, High, Strong, Cool, Change	Yes	**<Sunny, Warm, ?, Strong, ?, ?>**	Generalised

8. Formulae

Hypothesis representation	h = $<a_1, a_2, ..., a_n>$, where each $a_i \in$ {specific value, '?' (any value), Ø (no value)}
Initial hypothesis	$h_o = <Ø, Ø, ..., Ø>$ (most specific hypothesis)
Update rule (positive x only)	if $a_i = Ø$ then $a_i \leftarrow x_i$; else if $a_i \neq x_i$ then $a_i \leftarrow$ '?'
Matching function	$h(x) = 1$ iff $\forall i : (a_i =$ '?' or $a_i = x_i)$; otherwise $h(x) = 0$
Size of hypothesis space (EnjoySport)	Syntactically distinct = $5 \times 4^5 = 5120$; semantically distinct = $1 + 4 \times 3^5 = 973$

9. Python Program

File: `exp1_findS_basic.py`

```
"""
Experiment 1: FIND-S algorithm - basic implementation (EnjoySport dataset)
"""


# Attributes: Sky, AirTemp, Humidity, Wind, Water, Forecast | Target: EnjoySport
attributes = ["Sky", "AirTemp", "Humidity", "Wind", "Water", "Forecast"]

training_data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same",   "Yes"],
    ["Sunny", "Warm", "High",   "Strong", "Warm", "Same",   "Yes"],
    ["Rainy", "Cold", "High",   "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High",   "Strong", "Cool", "Change", "Yes"],
]



def find_s(data):
    """Return the most specific hypothesis consistent with the positive examples."""
    n_attr = len(data[0]) - 1
    hypothesis = ["0"] * n_attr          # most specific hypothesis <0,0,...,0>

    for example in data:
        if example[-1] == "Yes":         # consider positive examples only
            for i in range(n_attr):
                if hypothesis[i] == "0":         # first positive example
```

```
                    hypothesis[i] = example[i]
                elif hypothesis[i] != example[i]:  # mismatch -> generalise
                    hypothesis[i] = "?"
    return hypothesis



print("Training Data")
print("-" * 70)
print(attributes, "-> EnjoySport")
for row in training_data:
    print(row)

print("\nInitial Hypothesis :", ["0"] * len(attributes))
final_h = find_s(training_data)
print("Final Hypothesis   :", final_h)
```

10. Output

```
Training Data
----------------------------------------------------------------------
['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast'] -> EnjoySport
['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change', 'No']
['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']

Initial Hypothesis : ['0', '0', '0', '0', '0', '0']
Final Hypothesis   : ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

11. Result / Conclusion

Final Hypothesis

h = <Sunny, Warm, ?, Strong, ?, ?>

The FIND-S algorithm was implemented and executed successfully. The maximally specific hypothesis for the EnjoySport data is <Sunny, Warm, ?, Strong, ?, ?>, i.e. the sport is enjoyed whenever the sky is sunny, the air is warm and the wind is strong.

MACHINE LEARNING LABORATORY | EXPERIMENT 2 OF 5

FIND-S: Training Data Read from a CSV File (Student Placement)

Evaluation Component	Max. Marks	Marks Awarded
Aim & Concept	5	
Algorithm & Procedure	10	
Program (logic, readability)	15	
Output & Observation	10	
Result & Conclusion	5	
Viva-voce	5	
TOTAL	**50**	
Staff Signature with Date:		

1. Experiment No. & Date

Experiment No.	2 (Part 2 of 5)
Date	____ / ____ / _____
Topic	CSV input - Student placement

2. Experiment Question

> **Question**
> Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Part 2: CSV input - Student placement]

3. Aim

To implement the FIND-S algorithm so that it reads the training examples from a CSV file using the pandas library and finds the most specific hypothesis for the student placement data.

4. Concept

In real machine-learning work the training data is stored in files rather than hard-coded. The pandas library loads a CSV file into a DataFrame; the last column is treated as the target (class label) and all the other columns as attributes.

FIND-S then scans the DataFrame row by row. Rows labelled 'Yes' are used to specialise/generalise the hypothesis and rows labelled 'No' are skipped. The algorithm is independent of the number of attributes, so the same code works for any CSV file with the same layout.

Hypothesis symbols: '?' accepts any value; a specific value (e.g. Sunny) accepts only that value; '0' / ∅ accepts no value.

5. Dataset Description

Source	placement.csv
Attributes	CGPA, Internship, Communication, Projects
Target attribute	Placed (Yes = positive, No = negative)
Summary	9 instances, 4 attributes, 5 positive and 4 negative examples stored in placement.csv.

Training data *(shaded rows = positive examples, unshaded rows = negative examples)*

No.	CGPA	Internship	Communication	Projects	Placed
1	High	Yes	Good	Many	**Yes**
2	High	Yes	Good	Few	**Yes**
3	Low	No	Average	Few	**No**
4	High	Yes	Average	Many	**Yes**
5	Medium	No	Good	Few	**No**
6	High	Yes	Good	Many	**Yes**
7	Low	Yes	Average	Few	**No**
8	High	No	Good	Many	**Yes**
9	Medium	Yes	Average	Few	**No**

Contents of placement.csv

```
CGPA,Internship,Communication,Projects,Placed
High,Yes,Good,Many,Yes
High,Yes,Good,Few,Yes
Low,No,Average,Few,No
High,Yes,Average,Many,Yes
Medium,No,Good,Few,No
High,Yes,Good,Many,Yes
Low,Yes,Average,Few,No
High,No,Good,Many,Yes
Medium,Yes,Average,Few,No
```

6. Algorithm

Initialise h to the most specific hypothesis in H: h = <∅, ∅, ∅, ..., ∅> (written as '0' in the program).

For each training example x in the training data:

If x is a positive example (target = Yes), then for each attribute a_i in h:

(a) If h[i] = '0', set h[i] = x[i] (copy the first positive example).

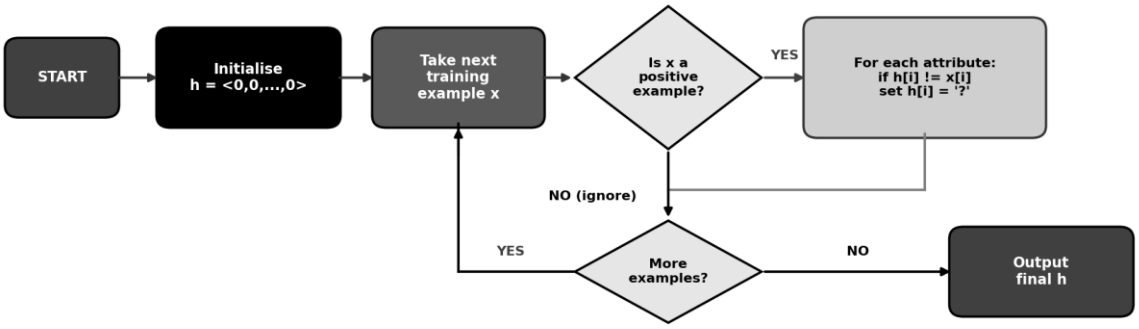
(b) Else if h[i] = x[i], leave h[i] unchanged.

(c) Else replace h[i] with '?' (generalise just enough to cover x).

If x is a negative example, ignore it (h is not changed).

Output the hypothesis h (the maximally specific hypothesis consistent with all positive examples).

FIND-S Algorithm - Flowchart



7. Step-by-step Procedure

1. Create placement.csv and place it in the same folder as the program; import pandas and load the file with pd.read_csv().
2. Initialise the hypothesis with the most specific value (all '0'), or copy the first positive example.
3. Take the training examples one by one. If an example is negative, skip it.
4. For a positive example, compare each attribute of h with the example: keep the value if they match, otherwise replace it with '?'.
5. Record the hypothesis after every example (see the trace table below).
6. Print the final hypothesis after all examples have been processed.

Hypothesis trace *(shaded rows = positive examples)*

Step	Instance	Label	Hypothesis h after step	Action
Initial	-	-	**<0, 0, 0, 0>**	-
x1	High, Yes, Good, Many	Yes	**<High, Yes, Good, Many>**	First positive - copy
x2	High, Yes, Good, Few	Yes	**<High, Yes, Good, ?>**	Generalised
x3	Low, No, Average, Few	No	**<High, Yes, Good, ?>**	Negative - ignored
x4	High, Yes, Average, Many	Yes	**<High, Yes, ?, ?>**	Generalised
x5	Medium, No, Good, Few	No	**<High, Yes, ?, ?>**	Negative - ignored
x6	High, Yes, Good, Many	Yes	**<High, Yes, ?, ?>**	Generalised
x7	Low, Yes, Average, Few	No	**<High, Yes, ?, ?>**	Negative - ignored
x8	High, No, Good, Many	Yes	**<High, ?, ?, ?>**	Generalised
x9	Medium, Yes, Average, Few	No	**<High, ?, ?, ?>**	Negative - ignored

8. Formulae

Hypothesis representation	$h = \langle a_1, a_2, ..., a_n \rangle$, where each $a_i \in$ {specific value, '?' (any value), $\emptyset$ (no value)}

Initial hypothesis	h_o = <∅, ∅, ..., ∅> (most specific hypothesis)
Update rule (positive x only)	if a_i = ∅ then $a_i \leftarrow x_i$; else if $a_i \neq x_i$ then $a_i \leftarrow$ '?'
Matching function	h(x) = 1 iff ∀i : (a_i = '?' or $a_i = x_i$); otherwise h(x) = 0
Class counts	Number of positives P = count(Placed = Yes); number of negatives N = count(Placed = No); total = P + N

9. Python Program

File: `exp2_findS_csv.py`

```
"""
Experiment 2: FIND-S algorithm - reading the training data from a CSV file
Dataset: placement.csv (keep it in the same folder as this program)
"""
import pandas as pd

data = pd.read_csv("placement.csv")
print("Training Data:\n")
print(data.to_string(index=False))

attributes = data.columns[:-1]            # all columns except the target
target = data.columns[-1]                 # last column = class label

print("\nNumber of instances  :", len(data))
print("Number of attributes :", len(attributes))
print("Positive examples    :", (data[target] == "Yes").sum())
print("Negative examples    :", (data[target] == "No").sum())



def find_s(df):
    hypothesis = None
    for _, row in df.iterrows():
        if row[target] == "Yes":                        # positive example
            values = [row[a] for a in attributes]
            if hypothesis is None:                      # first positive example
                hypothesis = values.copy()
            else:
                for i, v in enumerate(values):
                    if hypothesis[i] != v:
                        hypothesis[i] = "?"             # generalise
    return hypothesis



h = find_s(data)
print("\nMost specific hypothesis <", ", ".join(attributes), ">")
print("Final Hypothesis :", h)
```

10. Output

```
Training Data:

  CGPA Internship Communication Projects Placed
  High        Yes          Good     Many    Yes
  High        Yes          Good      Few    Yes
   Low         No       Average      Few     No
  High        Yes       Average     Many    Yes
Medium         No          Good      Few     No
  High        Yes          Good     Many    Yes
   Low        Yes       Average      Few     No
  High         No          Good     Many    Yes
Medium        Yes       Average      Few     No


Number of instances  : 10
Number of attributes : 4
Positive examples    : 6
Negative examples    : 4

Most specific hypothesis < CGPA , Internship , Communication , Projects >
Final Hypothesis : ['?', 'Yes', '?', '?']
```

```
Number of instances  : 9
Number of attributes : 4
Positive examples    : 5
Negative examples    : 4

Most specific hypothesis < CGPA, Internship, Communication, Projects >
Final Hypothesis : ['High', '?', '?', '?']
```

11. Result / Conclusion

Final Hypothesis

h = <High, ?, ?, ?>

The training data was loaded from placement.csv and FIND-S produced the hypothesis <High, ?, ?, ?>. A student with a High CGPA is predicted to be placed irrespective of internship, communication skill and project count.

MACHINE LEARNING LABORATORY | EXPERIMENT 3 OF 5

FIND-S: Step-by-Step Trace and Visualisation

Evaluation Component	Max. Marks	Marks Awarded
Aim & Concept	5	
Algorithm & Procedure	10	
Program (logic, readability)	15	
Output & Observation	10	
Result & Conclusion	5	
Viva-voce	5	
TOTAL	**50**	
Staff Signature with Date:		

1. Experiment No. & Date

Experiment No.	3 (Part 3 of 5)
Date	____ / ____ / _____
Topic	Step-by-step trace with graph

2. Experiment Question

> **Question**
> Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Part 3: Step-by-step trace with graph]

3. Aim

To demonstrate the working of the FIND-S algorithm by displaying the hypothesis after every training instance and to plot how the hypothesis becomes more general as positive examples are processed.

4. Concept

The behaviour of FIND-S is easiest to understand by tracing the hypothesis h_0, h_1, h_2, ... after each training example. The hypothesis can only move from specific towards general: attributes change from a concrete value to '?', never the other way round.

A graph of the number of '?' symbols in h after each step visualises this monotonic generalisation. Negative examples leave h unchanged, so they appear as flat segments on the graph.

Hypothesis symbols: '?' accepts any value; a specific value (e.g. Sunny) accepts only that value; '0' / ∅ accepts no value.

5. Dataset Description

Source	Hard-coded Python list
Attributes	Sky, AirTemp, Humidity, Wind, Water, Forecast
Target attribute	EnjoySport (Yes = positive, No = negative)
Summary	Same EnjoySport data as Experiment 1, used here to trace every step of the algorithm.

Training data *(shaded rows = positive examples, unshaded rows = negative examples)*

No.	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
2	Sunny	Warm	High	Strong	Warm	Same	**Yes**
3	Rainy	Cold	High	Strong	Warm	Change	**No**
4	Sunny	Warm	High	Strong	Cool	Change	**Yes**

6. Algorithm

Initialise h to the most specific hypothesis in H: h = <∅, ∅, ∅, ..., ∅> (written as '0' in the program).

For each training example x in the training data:

If x is a positive example (target = Yes), then for each attribute a_i in h:

(a) If h[i] = '0', set h[i] = x[i] (copy the first positive example).

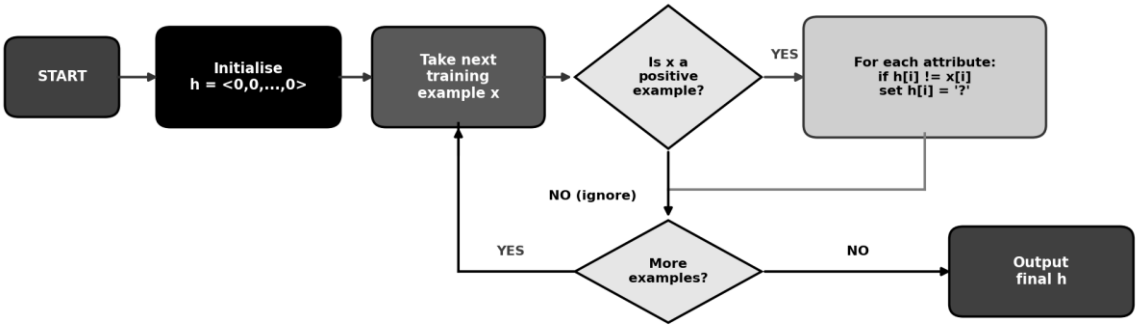
(b) Else if h[i] = x[i], leave h[i] unchanged.

(c) Else replace h[i] with '?' (generalise just enough to cover x).

If x is a negative example, ignore it (h is not changed).

Output the hypothesis h (the maximally specific hypothesis consistent with all positive examples).

FIND-S Algorithm - Flowchart



7. Step-by-step Procedure

1. Import the required modules and define the attribute names and the training data set.
2. Initialise the hypothesis with the most specific value (all '0'), or copy the first positive example.
3. Take the training examples one by one. If an example is negative, skip it.

4. For a positive example, compare each attribute of h with the example: keep the value if they match, otherwise replace it with '?'.
5. Record the hypothesis after every example (see the trace table below).
6. Count the '?' symbols after each step and plot the graph using matplotlib; save it as hypothesis_evolution.png.

Hypothesis trace *(shaded rows = positive examples)*

Step	Instance	Label	Hypothesis h after step	Action
Initial	-	-	**<0, 0, 0, 0, 0, 0>**	-
x1	Sunny, Warm, Normal, Strong, Warm, Same	Yes	**<Sunny, Warm, Normal, Strong, Warm, Same>**	First positive - copy
x2	Sunny, Warm, High, Strong, Warm, Same	Yes	**<Sunny, Warm, ?, Strong, Warm, Same>**	Generalised
x3	Rainy, Cold, High, Strong, Warm, Change	No	**<Sunny, Warm, ?, Strong, Warm, Same>**	Negative - ignored
x4	Sunny, Warm, High, Strong, Cool, Change	Yes	**<Sunny, Warm, ?, Strong, ?, ?>**	Generalised

8. Formulae

Hypothesis representation	$h = \langle a_1, a_2, ..., a_n \rangle$, where each $a_i \in$ {specific value, '?' (any value), $\emptyset$ (no value)}
Initial hypothesis	$h_o = \langle \emptyset, \emptyset, ..., \emptyset \rangle$ (most specific hypothesis)
Update rule (positive x only)	if $a_i = \emptyset$ then $a_i \leftarrow x_i$; else if $a_i \neq x_i$ then $a_i \leftarrow$ '?'
Matching function	$h(x) = 1$ iff $\forall i : (a_i =$ '?' or $a_i = x_i)$; otherwise $h(x) = 0$
Generalisation count	G_k = number of '?' symbols in hypothesis h_k. Property: $G_o \leq G_1 \leq ... \leq G_n$ (non-decreasing)

9. Python Program

File: `exp3_findS_trace.py`

```
"""
Experiment 3: FIND-S algorithm - step-by-step trace and visualisation
Shows the hypothesis after every training instance and plots how the
hypothesis becomes more general as positive examples are processed.
"""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

attributes = ["Sky", "AirTemp", "Humidity", "Wind", "Water", "Forecast"]
data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same",   "Yes"],
    ["Sunny", "Warm", "High",   "Strong", "Warm", "Same",   "Yes"],
    ["Rainy", "Cold", "High",   "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High",   "Strong", "Cool", "Change", "Yes"],
]

h = ["0"] * len(attributes)
history = [("Initial", h.copy(), "-")]
print("Initial hypothesis h0 =", h, "\n")

for k, ex in enumerate(data, start=1):
    label = ex[-1]
```

```
    print(f"Instance {k}: {ex[:-1]}  ->  {label}")
    if label == "Yes":
        for i in range(len(attributes)):
            if h[i] == "0":
                h[i] = ex[i]
            elif h[i] != ex[i]:
                print(f"   '{attributes[i]}' differs ({h[i]} vs {ex[i]}) -> replaced
by '?'")
                h[i] = "?"
    else:
        print("   Negative example -> ignored, hypothesis unchanged")
    history.append((f"x{k}", h.copy(), label))
    print(f"   h{k} = {h}\n")

print("Final Hypothesis =", h)

# ---- Visualisation: number of '?' (generalised attributes) after each step ----
steps = [s[0] for s in history]
general = [sum(1 for v in s[1] if v == "?") for s in history]

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(steps, general, color="black", linewidth=1.5, zorder=1)
for s, g, hist in zip(steps, general, history):
    if hist[2] == "Yes":      # positive example: filled circle
        ax.scatter(s, g, marker="o", s=140, c="black", zorder=2)
    elif hist[2] == "No":     # negative example: hollow square
        ax.scatter(s, g, marker="s", s=140, facecolors="white", edgecolors="black",
linewidths=1.8, zorder=2)
    else:                     # initial hypothesis: grey diamond
        ax.scatter(s, g, marker="D", s=120, c="grey", zorder=2)
    ax.annotate(str(g), (s, g), textcoords="offset points", xytext=(0, 12),
ha="center")
ax.set_xlabel("Training instance processed")
ax.set_ylabel("Number of '?' in hypothesis")
ax.set_title("FIND-S: Hypothesis generalisation")
ax.set_yticks(range(0, len(attributes) + 1))
ax.grid(alpha=0.3)
ax.legend(handles=[
    plt.Line2D([], [], marker="o", color="black", linestyle="", markersize=9,
label="Positive example"),
    plt.Line2D([], [], marker="s", markerfacecolor="white", markeredgecolor="black",
color="black", linestyle="", markersize=9, label="Negative example"),
    plt.Line2D([], [], marker="D", color="grey", linestyle="", markersize=8,
label="Initial hypothesis")], loc="lower right")
plt.tight_layout()
plt.savefig("hypothesis_evolution.png", dpi=200)
print("Graph saved as hypothesis_evolution.png")
```

10. Output

```
Initial hypothesis h0 = ['0', '0', '0', '0', '0', '0']


Instance 1: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']  ->  Yes
   h1 = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']


Instance 2: ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same']  ->  Yes
   'Humidity' differs (Normal vs High) -> replaced by '?'
   h2 = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']


Instance 3: ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change']  ->  No
   Negative example -> ignored, hypothesis unchanged
   h3 = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']


Instance 4: ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change']  ->  Yes
   'Humidity' differs (? vs High) -> replaced by '?'
   'Water' differs (Warm vs Cool) -> replaced by '?'
   'Forecast' differs (Same vs Change) -> replaced by '?'
   h4 = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

```
Final Hypothesis = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Graph saved as hypothesis_evolution.png
```

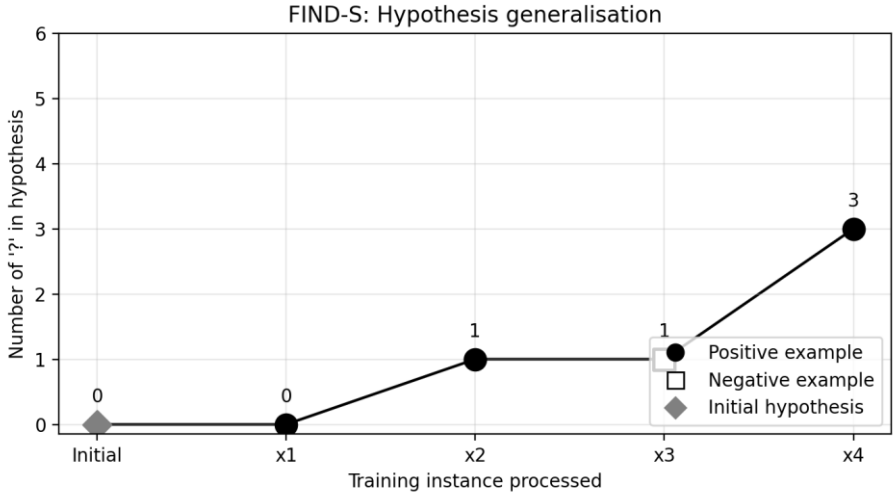


Fig: Generalisation of the hypothesis after each training instance

11. Result / Conclusion

Final Hypothesis

h = <Sunny, Warm, ?, Strong, ?, ?>

The step-by-step trace shows h_0 = <0,0,0,0,0,0> evolving to h_4 = <Sunny, Warm, ?, Strong, ?, ?>. The graph confirms that generalisation grows only on positive examples (x_2 and x_4) while the negative example x_3 leaves the hypothesis unchanged.

MACHINE LEARNING LABORATORY | EXPERIMENT 4 OF 5

FIND-S: Object-Oriented Implementation (Flu Diagnosis)

Evaluation Component	Max. Marks	Marks Awarded
Aim & Concept	5	
Algorithm & Procedure	10	
Program (logic, readability)	15	
Output & Observation	10	
Result & Conclusion	5	
Viva-voce	5	
TOTAL	**50**	
Staff Signature with Date:		

1. Experiment No. & Date

Experiment No.	4 (Part 4 of 5)
Date	____ / ____ / _____
Topic	Class-based implementation - Flu diagnosis

2. Experiment Question

> **Question**
> Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Part 4: Class-based implementation - Flu diagnosis]

3. Aim

To implement the FIND-S algorithm as a reusable Python class with fit() and predict() methods and to apply it to a medical (flu diagnosis) data set.

4. Concept

Structuring a learning algorithm as a class (fit / predict) is the standard pattern used by machine-learning libraries. fit() learns the maximally specific hypothesis from the training data, and predict() classifies a new instance using the learned hypothesis.

A new instance is classified as positive (Flu = Yes) only if it satisfies every constraint in the hypothesis; the symbol '?' accepts any value. Instances that violate any specific constraint are classified as negative.

Hypothesis symbols: '?' accepts any value; a specific value (e.g. Sunny) accepts only that value; '0' / ∅ accepts no value.

5. Dataset Description

Source	Python lists X and y
Attributes	Fever, Cough, Headache, BodyAche, Fatigue
Target attribute	Flu (Yes = positive, No = negative)
Summary	6 instances, 5 attributes, 3 positive and 3 negative examples.

Training data *(shaded rows = positive examples, unshaded rows = negative examples)*

No.	Fever	Cough	Headache	BodyAche	Fatigue	Flu
1	High	Yes	Yes	Yes	Yes	**Yes**
2	High	Yes	No	Yes	Yes	**Yes**
3	Normal	No	No	No	No	**No**
4	High	Yes	Yes	Yes	No	**Yes**
5	Mild	No	Yes	No	Yes	**No**
6	High	No	No	No	No	**No**

6. Algorithm

Initialise h to the most specific hypothesis in H: h = <∅, ∅, ∅, ..., ∅> (written as '0' in the program).
For each training example x in the training data:

If x is a positive example (target = Yes), then for each attribute a_i in h:

(a) If h[i] = '0', set h[i] = x[i] (copy the first positive example).

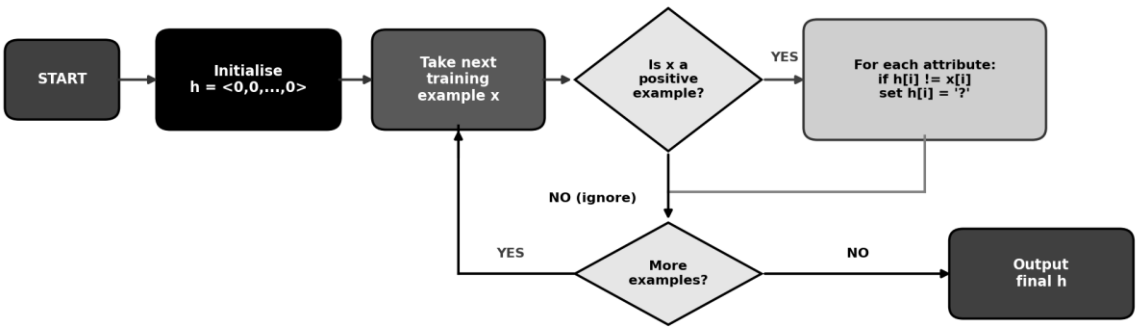
(b) Else if h[i] = x[i], leave h[i] unchanged.

(c) Else replace h[i] with '?' (generalise just enough to cover x).

If x is a negative example, ignore it (h is not changed).

Output the hypothesis h (the maximally specific hypothesis consistent with all positive examples).

FIND-S Algorithm - Flowchart



7. Step-by-step Procedure

1. Define the class FindS with the methods fit() and predict_one(), and store the training features X and labels y.

2. Initialise the hypothesis with the most specific value (all '0'), or copy the first positive example.
3. Take the training examples one by one. If an example is negative, skip it.
4. For a positive example, compare each attribute of h with the example: keep the value if they match, otherwise replace it with '?'.
5. Record the hypothesis after every example (see the trace table below).
6. Call predict_one() on new patient records to classify them with the learned hypothesis.

Hypothesis trace *(shaded rows = positive examples)*

Step	Instance	Label	Hypothesis h after step	Action
Initial	-	-	**<0, 0, 0, 0, 0>**	-
x1	High, Yes, Yes, Yes, Yes	Yes	**<High, Yes, Yes, Yes, Yes>**	First positive - copy
x2	High, Yes, No, Yes, Yes	Yes	**<High, Yes, ?, Yes, Yes>**	Generalised
x3	Normal, No, No, No, No	No	**<High, Yes, ?, Yes, Yes>**	Negative - ignored
x4	High, Yes, Yes, Yes, No	Yes	**<High, Yes, ?, Yes, ?>**	Generalised
x5	Mild, No, Yes, No, Yes	No	**<High, Yes, ?, Yes, ?>**	Negative - ignored
x6	High, No, No, No, No	No	**<High, Yes, ?, Yes, ?>**	Negative - ignored

8. Formulae

Hypothesis representation	h = $<a_1, a_2, ..., a_n>$, where each $a_i \in$ {specific value, '?' (any value), Ø (no value)}
Initial hypothesis	$h_o = <Ø, Ø, ..., Ø>$ (most specific hypothesis)
Update rule (positive x only)	if $a_i = Ø$ then $a_i \leftarrow x_i$; else if $a_i \neq x_i$ then $a_i \leftarrow$ '?'
Matching function	$h(x) = 1$ iff $\forall i : (a_i =$ '?' or $a_i = x_i)$; otherwise $h(x) = 0$
Prediction	$\hat{y}(x)$ = Yes if $h(x) = 1$, otherwise No

9. Python Program

File: `exp4_findS_class.py`

```
"""
Experiment 4: FIND-S algorithm - object-oriented implementation
Dataset: Flu diagnosis (medical domain)
"""



class FindS:
    def __init__(self, attributes):
        self.attributes = attributes
        self.hypothesis = None

    def fit(self, X, y, positive="Yes"):
        for features, label in zip(X, y):
            if label != positive:
                continue                                # ignore negatives
```

```
            if self.hypothesis is None:
                self.hypothesis = list(features)         # most specific
            else:
                self.hypothesis = [h if h == f else "?"
                                   for h, f in zip(self.hypothesis, features)]
        return self

    def predict_one(self, features):
        return "Yes" if all(h == "?" or h == f
                            for h, f in zip(self.hypothesis, features)) else "No"


attributes = ["Fever", "Cough", "Headache", "BodyAche", "Fatigue"]
X = [
    ["High",   "Yes", "Yes", "Yes", "Yes"],
    ["High",   "Yes", "No",  "Yes", "Yes"],
    ["Normal", "No",  "No",  "No",  "No"],
    ["High",   "Yes", "Yes", "Yes", "No"],
    ["Mild",   "No",  "Yes", "No",  "Yes"],
    ["High",   "No",  "No",  "No",  "No"],
]
y = ["Yes", "Yes", "No", "Yes", "No", "No"]

print("Flu Diagnosis - Training Data")
print(attributes, "-> Flu")
for row, label in zip(X, y):
    print(row, "->", label)

model = FindS(attributes).fit(X, y)
print("\nFinal Hypothesis :", model.hypothesis)

print("\nPredictions for new patients:")
new_patients = [
    ["High", "Yes", "No",  "Yes", "No"],
    ["High", "No",  "Yes", "Yes", "Yes"],
]
for p in new_patients:
    print(p, "->", model.predict_one(p))
```

10. Output

```
Flu Diagnosis - Training Data
['Fever', 'Cough', 'Headache', 'BodyAche', 'Fatigue'] -> Flu
['High', 'Yes', 'Yes', 'Yes', 'Yes'] -> Yes
['High', 'Yes', 'No', 'Yes', 'Yes'] -> Yes
['Normal', 'No', 'No', 'No', 'No'] -> No
['High', 'Yes', 'Yes', 'Yes', 'No'] -> Yes
['Mild', 'No', 'Yes', 'No', 'Yes'] -> No
['High', 'No', 'No', 'No', 'No'] -> No

Final Hypothesis : ['High', 'Yes', '?', 'Yes', '?']

Predictions for new patients:
['High', 'Yes', 'No', 'Yes', 'No'] -> Yes
['High', 'No', 'Yes', 'Yes', 'Yes'] -> No
```

11. Result / Conclusion

Final Hypothesis

h = <High, Yes, ?, Yes, ?>

The FindS class learned the hypothesis <High, Yes, ?, Yes, ?> for flu diagnosis: a patient with high fever, cough and body ache is diagnosed with flu. Two new patients were classified correctly using predict().

MACHINE LEARNING LABORATORY | EXPERIMENT 5 OF 5

FIND-S: Prediction, Consistency Check and Limitations (Loan Approval)

Evaluation Component	Max. Marks	Marks Awarded
Aim & Concept	5	
Algorithm & Procedure	10	
Program (logic, readability)	15	
Output & Observation	10	
Result & Conclusion	5	
Viva-voce	5	
TOTAL	**50**	
Staff Signature with Date:		

1. Experiment No. & Date

Experiment No.	5 (Part 5 of 5)
Date	____ / ____ / _____
Topic	Prediction, accuracy and limitations

2. Experiment Question

> **Question**
> Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. [Part 5: Prediction, accuracy and limitations]

3. Aim

To apply the learned FIND-S hypothesis to classify unseen test instances, verify its consistency with the negative training examples, compute accuracy, and demonstrate the limitation of FIND-S on noisy data.

4. Concept

After learning, a hypothesis is useful only if it can classify new data. A hypothesis is consistent if it covers all positive examples and none of the negative examples seen during training. FIND-S guarantees that its output is the most specific hypothesis consistent with the positive examples, but consistency with negative examples holds only when the target concept lies in H and the data is noise-free.

Limitations: (i) FIND-S cannot tell whether it has found the only consistent hypothesis, (ii) it ignores negative examples, and (iii) a single mislabelled positive example can over-generalise the hypothesis to <?, ?, ..., ?>.

Hypothesis symbols: '?' accepts any value; a specific value (e.g. Sunny) accepts only that value; '0' / ∅ accepts no value.

5. Dataset Description

Source	Python lists train and test
Attributes	Income, Credit, Employment, Property
Target attribute	Approved (Yes = positive, No = negative)
Summary	Training set: 6 instances (4 attributes). A separate test set of 4 labelled instances is used for evaluation (shown below).

Training data *(shaded rows = positive examples, unshaded rows = negative examples)*

No.	Income	Credit	Employment	Property	Approved
1	High	Good	Salaried	Owned	**Yes**
2	High	Good	Self-employed	Owned	**Yes**
3	Low	Bad	Salaried	Rented	**No**
4	High	Good	Salaried	Rented	**Yes**
5	Medium	Bad	Unemployed	Rented	**No**
6	Medium	Good	Salaried	Owned	**Yes**

Test data *(unseen instances with known labels)*

No.	Income	Credit	Employment	Property	Approved
1	Low	Good	Salaried	Owned	**Yes**
2	High	Bad	Salaried	Owned	**No**
3	Medium	Good	Unemployed	Rented	**Yes**
4	Low	Good	Self-employed	Rented	**No**

6. Algorithm

Initialise h to the most specific hypothesis in H: h = <∅, ∅, ∅, ..., ∅> (written as '0' in the program).
For each training example x in the training data:

If x is a positive example (target = Yes), then for each attribute a_i in h:

(a) If h[i] = '0', set h[i] = x[i] (copy the first positive example).

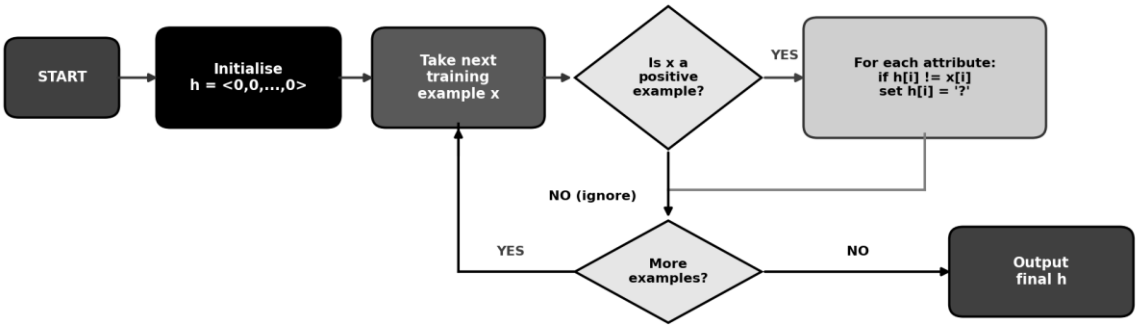
(b) Else if h[i] = x[i], leave h[i] unchanged.

(c) Else replace h[i] with '?' (generalise just enough to cover x).

If x is a negative example, ignore it (h is not changed).

Output the hypothesis h (the maximally specific hypothesis consistent with all positive examples).

FIND-S Algorithm - Flowchart



7. Step-by-step Procedure

1. Import the required modules and define the attribute names and the training data set.
2. Initialise the hypothesis with the most specific value (all '0'), or copy the first positive example.
3. Take the training examples one by one. If an example is negative, skip it.
4. For a positive example, compare each attribute of h with the example: keep the value if they match, otherwise replace it with '?'.
5. Record the hypothesis after every example (see the trace table below).
6. Check consistency with all negative examples, classify the test instances, compute accuracy, and add a noisy example to observe the limitation.

Hypothesis trace *(shaded rows = positive examples)*

Step	Instance	Label	Hypothesis h after step	Action
Initial	-	-	**<0, 0, 0, 0>**	-
x1	High, Good, Salaried, Owned	Yes	**<High, Good, Salaried, Owned>**	First positive - copy
x2	High, Good, Self-employed, Owned	Yes	**<High, Good, ?, Owned>**	Generalised
x3	Low, Bad, Salaried, Rented	No	**<High, Good, ?, Owned>**	Negative - ignored
x4	High, Good, Salaried, Rented	Yes	**<High, Good, ?, ?>**	Generalised
x5	Medium, Bad, Unemployed, Rented	No	**<High, Good, ?, ?>**	Negative - ignored
x6	Medium, Good, Salaried, Owned	Yes	**<?, Good, ?, ?>**	Generalised

8. Formulae

Hypothesis representation	$h = \langle a_1, a_2, ..., a_n \rangle$, where each $a_i \in$ {specific value, '?' (any value), $\emptyset$ (no value)}
Initial hypothesis	$h_o = \langle \emptyset, \emptyset, ..., \emptyset \rangle$ (most specific hypothesis)
Update rule (positive x only)	if $a_i = \emptyset$ then $a_i \leftarrow x_i$; else if $a_i \neq x_i$ then $a_i \leftarrow$ '?'
Matching function	$h(x) = 1$ iff $\forall i : (a_i =$ '?' or $a_i = x_i)$; otherwise $h(x) = 0$

Accuracy	Accuracy (%) = (number of correct predictions / total test instances) × 100
Consistency	Consistent(h) ⇔ ∀ <x, c(x)> ∈ D : h(x) = c(x)

9. Python Program

File: `exp5_findS_predict.py`

```
"""
Experiment 5: FIND-S algorithm - classification of unseen instances,
consistency check and limitations.   Dataset: Loan approval
"""
attributes = ["Income", "Credit", "Employment", "Property"]

train = [
    (["High",   "Good", "Salaried",      "Owned"],  "Yes"),
    (["High",   "Good", "Self-employed", "Owned"],  "Yes"),
    (["Low",    "Bad",  "Salaried",      "Rented"], "No"),
    (["High",   "Good", "Salaried",      "Rented"], "Yes"),
    (["Medium", "Bad",  "Unemployed",    "Rented"], "No"),
    (["Medium", "Good", "Salaried",      "Owned"],  "Yes"),
]
test = [
    (["Low",    "Good", "Salaried",      "Owned"],  "Yes"),
    (["High",   "Bad",  "Salaried",      "Owned"],  "No"),
    (["Medium", "Good", "Unemployed",    "Rented"], "Yes"),
    (["Low",    "Good", "Self-employed", "Rented"], "No"),
]



def find_s(data):
    h = None
    for x, label in data:
        if label == "Yes":
            h = x.copy() if h is None else [a if a == b else "?" for a, b in zip(h,
x)]
    return h



def classify(h, x):
    return "Yes" if all(a == "?" or a == b for a, b in zip(h, x)) else "No"



h = find_s(train)
print("Final Hypothesis :", h)

# 1. Consistency check: hypothesis must not cover any negative training example
consistent = all(classify(h, x) == "No" for x, label in train if label == "No")
print("\nConsistent with all negative training examples? :", consistent)

# 2. Classify unseen test instances
print("\n{:<50}{:<10}{:<10}{}".format("Test instance", "Actual", "Predicted",
"Result"))
print("-" * 80)
correct = 0
for x, actual in test:
    pred = classify(h, x)
    ok = pred == actual
    correct += ok
    print("{:<50}{:<10}{:<10}{}".format(str(x), actual, pred, "Correct" if ok else
"Wrong"))

print(f"\nAccuracy on test data = {correct}/{len(test)} =
{100*correct/len(test):.0f}%")
```

```
# 3. Limitation: one noisy (wrongly labelled) positive example
noisy = train + [(["Low", "Bad", "Unemployed", "Rented"], "Yes")]
print("\nHypothesis after adding one noisy positive example:", find_s(noisy))
print("The hypothesis collapses to all '?' -> FIND-S cannot handle noisy data.")
```

10. Output

```
Final Hypothesis : ['?', 'Good', '?', '?']

Consistent with all negative training examples? : True

Test instance                                        Actual    Predicted Result
-------------------------------------------------------------------------------
['Low', 'Good', 'Salaried', 'Owned']                 Yes       Yes       Correct
['High', 'Bad', 'Salaried', 'Owned']                 No        No        Correct
['Medium', 'Good', 'Unemployed', 'Rented']           Yes       Yes       Correct
['Low', 'Good', 'Self-employed', 'Rented']           No        Yes       Wrong

Accuracy on test data = 3/4 = 75%

Hypothesis after adding one noisy positive example: ['?', '?', '?', '?']
The hypothesis collapses to all '?' -> FIND-S cannot handle noisy data.
```

11. Result / Conclusion

Final Hypothesis

h = <?, Good, ?, ?>

FIND-S learned <?, Good, ?, ?> (loan approved when the credit rating is Good). The hypothesis was consistent with all negative training examples and achieved 75% accuracy (3 of 4) on the test set. The final experiment also showed that one noisy positive example collapses the hypothesis to <?, ?, ?, ?>, which is the main limitation of FIND-S.